UNIVERSIDAD DE ORIENTE

FACULTAD DE INGENIERÍA Y ARQUITECTURA

GESTIÓN DE SERVIDORES WEB

CICLO 02-2026

LABORATORIO

UNIDAD 2 — SERVIDORES WEB PROPIETARIO Y LIBRE DISTRIBUCIÓN

CASO: "DULCE HOGAR" — REPOSTERÍA ARTESANAL

Equipo N.º: ___3___
Sección: __A___

Integrantes:
1. Julio Román Ventura Gómez
2. Oscar Fabricio Guevara Aguilar
3. Jorge Humberto Lara Rodezno
4. Jonathan David De la O Echeverria
5. Anderson Ezequiel Reyes Rivera
6. Isvi José Castillo Cruz

Docente: Ing. Alberto Turcios Ortez

Fecha de entrega: lunes 12 de Octubre

Índice

INTRODUCCIÓN 3
FASE 1 - DIMENSIONAMIENTO COLABORATIVO 3
FASE 2 - INSTALACIÓN DE APACHE 5
COMANDOS EJECUTADOS (WSL2 / UBUNTU) 5
RESULTADOS OBTENIDOS 5
FASE 3 - VIRTUAL HOST + REVERSE PROXY 7
COMANDOS EJECUTADOS (WSL2 / UBUNTU) 7
CONFIGURACIÓN: INDEX.JS (BACKEND EXPRESS) 8
CONFIGURACIÓN: DULCEHOGAR.SERVICE (SYSTEMD) 8
CONFIGURACIÓN: DULCEHOGAR.LOCAL.CONF (VIRTUAL HOST) 9
RESULTADOS OBTENIDOS 9
FASE 4 - HTTPS CON MKCERT 11
COMANDOS EJECUTADOS (WSL2 / UBUNTU) 11
CONFIGURACIÓN: PORTS.CONF 12
CONFIGURACIÓN: DULCEHOGAR.LOCAL-SSL.CONF 12
RESULTADOS OBTENIDOS 12
REPOSITORIO GITHUB 15
ESTRUCTURA PROPUESTA DEL REPOSITORIO 15
CONCLUSIONES 15
ROL DEL DOCUMENTADOR / QA 16
A. CAPTURAS DE EVIDENCIA 16
B. PRUEBAS DE INTEGRACIÓN 16
C. VALIDACIONES DE HTTPS 17

INTRODUCCIÓN

El presente informe documenta el desarrollo del Laboratorio 2 de la asignatura Gestión de Servidores Web, realizado por el Equipo 03 de la Universidad de Oriente. El caso de estudio corresponde a Dulce Hogar, una repostería artesanal que requiere publicar su catálogo de productos y una API de cotizaciones de manera segura y con capacidad de atender picos de demanda en fechas festivas.

El trabajo se organizó en cuatro fases consecutivas. En la Fase 1 se dimensionó la infraestructura necesaria y se decidió entre una alternativa en la nube y un servidor local. En la Fase 2 se instaló y verificó el servidor web Apache sobre WSL2 con Ubuntu. En la Fase 3 se configuró un Virtual Host con Reverse Proxy hacia un backend Express que corre en el puerto 9000 y se gestiona mediante systemd. Finalmente, en la Fase 4 se habilitó HTTPS utilizando certificados emitidos por una Autoridad Certificadora local creada con mkcert.

El rol del redactor de este informe es Documentador / QA, con responsabilidad sobre las capturas de evidencia, la redacción de los entregables, las pruebas de integración entre Apache y Express, y la validación del funcionamiento de HTTPS en el navegador. Cada fase incluye los comandos ejecutados, las configuraciones aplicadas, los resultados obtenidos y las respuestas a las preguntas guía del laboratorio.

Entorno de trabajo

Componente	Detalle
Sistema anfitrión	Windows con WSL2
Distribución	Ubuntu (WSL2)
Servidor Wev	Apache 2 (apache2)
Backend	Node.js con Express, puerto 9000, servicio systemd dulcehogar
Dominio local	dulcehogar.local (archivo hosts de Windows y de WSL)
Puertos	80 (HTTP), 8900 (HTTPS), 9000 (backend interno)
Certificados	mkcert v1.4.4 (CA local)

FASE 1 - DIMENSIONAMIENTO COLABORATIVO

1. ¿Los 2 vCPU y 4 GB de RAM son suficientes para el tráfico diario normal? ¿Y para el pico de 3,000 visitas? Calculen el factor pico aproximado.

Para el día normal sí alcanzan, con mucho margen. Para el pico alcanzan para contenido estático, pero con menos margen, que conviene confirmar con una prueba de carga antes de un día festivo.

Factor pico: 3,000 / 500 = 6. En festivos el tráfico es 6 veces el normal.

Día normal: una visita al catálogo genera unas 30 peticiones (HTML, CSS, JS e imágenes). 500 visitas son unas 15,000 peticiones al día, es decir, unas 0.17 por segundo en promedio. Los recursos sobran.

Pico: 3,000 visitas son unas 90,000 peticiones al día. Si se concentran en 2 a 4 horas, serían entre 6 y 12 por segundo, con ráfagas de 20 o más. Apache y el backend Express comparten los mismos 2 vCPU y 4

GB, por lo que el margen real es menor. Como referencia, en la Clase 20 medimos la API a través del proxy: 500 peticiones sin fallos y ~14 ms de promedio. No es una prueba del catálogo estático, por eso antes del festivo mediríamos con carga real.

2. ¿Recomendarían escalar vertical u horizontalmente para el pico? Justifiquen según costo, tiempo y presupuesto de una PyME.

Recomendamos escalar **verticalmente y de forma temporal**.

- **Costo:** subir de plan solo se paga durante los pocos días de pico al año. El escalado horizontal requiere un balanceador que se paga de forma continua, además de varios servidores.
- **Tiempo:** el vertical se aplica en minutos sin cambiar la arquitectura, aunque en muchos proveedores implica un reinicio breve, por eso se hace antes del festivo y no durante. El horizontal exige configurar el balanceador y que el backend pueda compartir sus datos entre servidores.

Presupuesto: una PyME como Dulce Hogar no cuenta con personal de TI para administrar varios servidores (el auto-scaling existe en nubes grandes, pero requiere conocimientos que no tiene). Lo complementamos con caché de archivos estáticos para reducir la carga.

3. ¿Qué recurso vigilarían primero si el servidor empieza a responder con lentitud? Expliquen con base en los 4 recursos (CPU, RAM, disco, red).

Empezaríamos por la RAM: con solo 4 GB compartidos entre el sistema, Apache y Express, si se agota el sistema usa swap y el rendimiento cae de golpe. En la práctica top y free -h muestran CPU, RAM y swap a la vez, y el primer recurso saturado es el cuello de botella.

Orden	Recurso	Comando	Justificacion
1	RAM	`free -h`	Es el recurso más escaso y frena todo si se agota.
2	CPU	`top`	Con solo 2 vCPU, un pico de peticiones puede saturarla.
3	Red	`ss -s`	El catálogo lleva fotos de pasteles; si no están optimizadas, la red puede saturarse antes que la CPU.
4	Disco	`df -h`	Un SSD rara vez es el cuello de botella, y Ubuntu rota los logs de Apache por defecto.

4. Decisión del equipo: ¿nube o servidor local? Presenten una tabla comparativa con 4 criterios.

Decisión: elegimos NUBE (VPS). De los 4 criterios, 3 favorecen a la nube. Solo el control favorece al servidor local, pero para una PyME como Dulce Hogar pesa menos que el costo y la capacidad de soportar los picos de 3,000 visitas. Recomendamos configurar alertas de presupuesto y comparar el costo a 3 a 5 años, eligiendo un VPS con transferencia de datos incluida.

Criterio	Nube (VPS)	Servidor local
Costo inicial	Bajo: se paga cada mes y no se compra equipo.	Alto: hay que comprar el hardware necesario.
Escalabilidad	Rápida: se amplía el plan en minutos, ideal para los picos festivos.	Lenta: hay que mejorar o comprar hardware, con más gasto y más tiempo.
Mantenimiento	A cargo del proveedor (hardware, energía y red).	A cargo de la empresa, con su propio personal y costo.
Control	Menos control físico sobre el equipo.	Control total sobre el equipo y los datos.

FASE 2 - INSTALACIÓN DE APACHE

COMANDOS EJECUTADOS (WSL2 / UBUNTU)

```
sudo apt update
sudo apt install -y apache2
sudo systemctl status apache2
sudo systemctl enable apache2
curl http://localhost
ls /etc/apache2/sites-available/
ls /var/www/html/
sudo apache2ctl configtest
```

RESULTADOS OBTENIDOS

- Apache instalado y activo: active (running).
- Página por defecto: "Apache2 Ubuntu Default Page - It works!".
- configtest: Syntax OK.

1. ¿Qué muestra la página por defecto de Apache? Capturen pantalla.

Muestra “Apache2 Ubuntu Default Page” con el mensaje “It works!", el logo de Ubuntu y un texto explicativo que confirma que el servidor Apache está instalado y funcionando correctamente. Esta página se sirve desde /var/www/html/index.html y es la forma más rápida de verificar que el servidor está escuchando en el puerto 80.

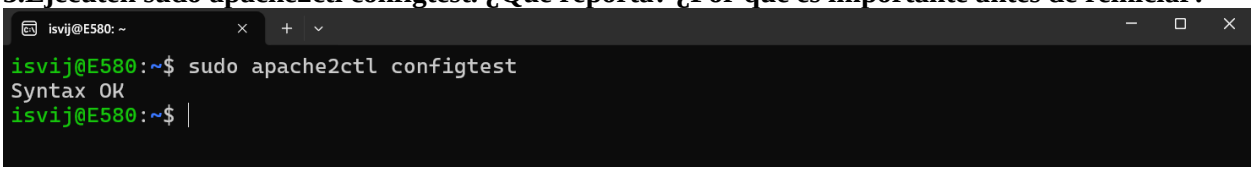
Captura: fase2-apache-activo.png



2. ¿Qué diferencia hay entre sites-available y sites-enabled?

El directorio sites-available contiene los archivos de configuración de los sitios web que se pueden habilitar en Apache. Por su parte, sites-enabled contiene normalmente enlaces simbólicos a las configuraciones de los sitios que están habilitados. Esto permite mantener configurados varios sitios web y activar o desactivar cada uno sin necesidad de eliminar sus archivos de configuración.

3.Ejecuten sudo apache2ctl configtest. ¿Qué reporta? ¿Por qué es importante antes de reiniciar?



Reporta "Syntax OK", lo que significa que la sintaxis de todos los archivos de configuración de Apache es correcta. Es importante porque permite detectar errores de configuración antes de recargar o reiniciar Apache, reduciendo el riesgo de interrupciones del servicio.

4. Reto: si Apache no arrancara, ¿qué 3 pasos seguirían para diagnosticar el problema?

Paso 1. Revisar el estado del servicio:

```
sudo systemctl status apache2
```

Permite comprobar si Apache está activo y consultar posibles mensajes de error.

Paso 2. Verificar la configuración:

```
sudo apache2ctl configtest
```

Permite detectar errores de sintaxis en los archivos de configuración.

Paso 3. Comprobar si el puerto 80 está ocupado:

```
sudo ss -tlnp | grep :80
```

Permite identificar si algún proceso está escuchando en el puerto 80, lo que podría impedir que Apache utilice ese puerto.

FASE 3 - VIRTUAL HOST + REVERSE PROXY

COMANDOS EJECUTADOS (WSL2 / UBUNTU)

```
# 1. Crear carpeta del sitio
sudo mkdir -p /var/www/dulcehogar.local
sudo chown -R www-data:www-data /var/www/dulcehogar.local
echo "<h1>Dulce Hogar</h1>" | sudo tee /var/www/dulcehogar.local/index.html

# 2. Crear contenedor Express
mkdir ~/dulcehogar-api
cd ~/dulcehogar-api
npm init -y
npm install express
nano index.js

# 3. Registrar servicio systemd
sudo nano /etc/systemd/system/dulcehogar.service
sudo systemctl daemon-reload
sudo systemctl enable dulcehogar
sudo systemctl start dulcehogar
sudo systemctl status dulcehogar

# 4. Habilitar módulos proxy
sudo a2enmod proxy proxy_http headers
sudo systemctl restart apache2

# 5. Crear Virtual Host
sudo nano /etc/apache2/sites-available/dulcehogar.local.conf
sudo a2ensite dulcehogar.local.conf
sudo apache2ctl configtest
sudo systemctl reload apache2
echo "127.0.0.1 dulcehogar.local" | sudo tee -a /etc/hosts

# 6. Windows PowerShell (Administrador)
Add-Content C:\Windows\System32\drivers\etc\hosts "127.0.0.1 dulcehogar.local"
Add-Content C:\Windows\System32\drivers\etc\hosts "::1 dulcehogar.local"

# 7. Probar
curl "http://dulcehogar.local/api/saludo?nombre=Equipo"
curl
"http://dulcehogar.local/api/cotizacion?nombre=Isvi&telefono=7777&producto=pastel
"
```

CONFIGURACIÓN: INDEX.JS (BACKEND EXPRESS)

```
const express = require('express');
const app = express();
const PORT = 9000;

app.get('/api/saludo', (req, res) => {
  const nombre = req.query.nombre || 'Mundo';
  res.json({ mensaje: `Hola, ${nombre}`, desde: 'dulcehogar:9000' });
});

app.get('/api/cotizacion', (req, res) => {
  const { nombre, telefono, producto } = req.query;
  res.json({
    cotizacionId: Date.now(),
    nombre: nombre || 'Sin nombre',
    telefono: telefono || 'Sin telefono',
    producto: producto || 'Sin producto',
    fecha: new Date().toISOString().split('T')[0],
    estado: 'pendiente'
  });
});

app.listen(PORT, () => {
  console.log(`dulcehogar escuchando en http://localhost:${PORT}`);
});
```

CONFIGURACIÓN: DULCEHOGAR.SERVICE (SYSTEMD)

```
[Unit]
Description=Dulce Hogar API
After=network.target

[Service]
Type=simple
User=isvij
WorkingDirectory=/home/isvij/dulcehogar-api
ExecStart=/usr/bin/node /home/isvij/dulcehogar-api/index.js
Restart=on-failure
RestartSec=10
StandardOutput=append:/home/isvij/dulcehogar-api/out.log
StandardError=append:/home/isvij/dulcehogar-api/err.log

[Install]
WantedBy=multi-user.target
```

CONFIGURACIÓN: DULCEHOGAR.LOCAL.CONF (VIRTUAL HOST)

```
<VirtualHost *:80>
    ServerName dulcehogar.local
    DocumentRoot /var/www/dulcehogar.local

    ProxyPreserveHost On
    ProxyPass /api http://localhost:9000/api
    ProxyPassReverse /api http://localhost:9000/api

    ErrorLog ${APACHE_LOG_DIR}/dulcehogar.local-error.log
    CustomLog ${APACHE_LOG_DIR}/dulcehogar.local-access.log combined
</VirtualHost>
```

RESULTADOS OBTENIDOS

- Contenedor Express corriendo en :9000 (systemd: active).
- Virtual Host habilitado (a2ensite exitoso).
- configtest: Syntax OK.
- curl vía Apache: {"mensaje":"Hola, Equipo","desde":"dulcehogar:9000"}

1. ¿Qué hace ProxyPass /api http://localhost:9000/api? Expliquen con sus palabras.

Reenvía todas las peticiones que empiecen con /api a la aplicación backend Express que corre en localhost:9000. Cuando el cliente pide http://dulcehogar.local/api/saludo, Apache no procesa esa petición, sino que la transfiere internamente a http://localhost:9000/api/saludo, recibe la respuesta del contenedor y se la devuelve al cliente como si fuera suya. El cliente nunca sabe que hay dos servidores trabajando.

2. ¿Qué diferencia hay entre ProxyPass y ProxyPassReverse?

ProxyPass se encarga de reenviar las peticiones del cliente desde Apache hacia el backend Express, mientras que ProxyPassReverse ajusta determinadas cabeceras de respuesta del backend, especialmente Location, para que las redirecciones utilicen la dirección del proxy en lugar de la dirección interna del backend.

Dicho de otra manera, ProxyPass indica "envía esta petición al backend" y ProxyPassReverse permite que, cuando el backend responda con una redirección, Apache ajuste la dirección correspondiente para que el cliente continúe accediendo mediante el dominio configurado.

De esta forma, ambas directivas facilitan la comunicación entre Apache y Express sin que el cliente tenga que acceder directamente al puerto 9000.

3. Reto: ¿qué pasaría si configtest falla? ¿Cómo lo depurarían entre todos?

Si configtest falla, significa que Apache ha detectado un problema en la configuración analizada. Antes de recargar o reiniciar el servicio, se debe identificar y corregir el error. Si Apache ya estaba funcionando, podría continuar utilizando la configuración anterior, dependiendo de la situación.

Pasos para depurar el problema:

1. Ejecutar **sudo apache2ctl configtest** y leer el mensaje de error.
2. Revisar el archivo de configuración indicado, prestando atención a las directivas, los nombres de los módulos y el cierre de las etiquetas, como </VirtualHost>.
3. Verificar que los módulos necesarios estén habilitados con sudo a2enmod proxy proxy_http headers.
4. Volver a ejecutar **sudo apache2ctl configtest** y, cuando indique Syntax OK, recargar Apache con **sudo systemctl reload apache2**.

4. Capture la respuesta del curl. ¿Por qué es la prueba de que las Unidades 1 y 2 están conectadas?

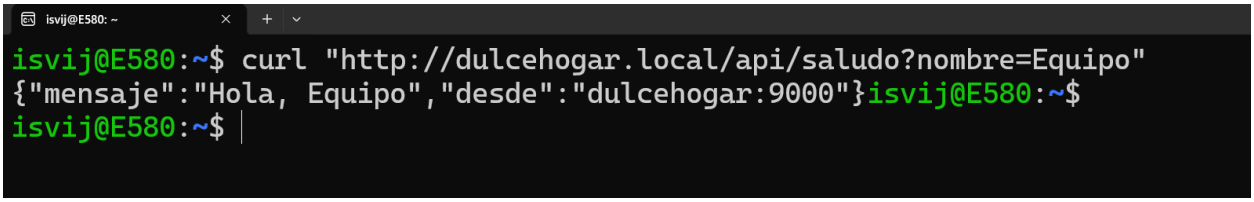
La respuesta JSON obtenida mediante curl es una evidencia de la integración entre Apache y Express. La petición se realiza a través del dominio dulcehogar.local, que utiliza el puerto 80 de Apache, y se reenvía al backend Express, que escucha en el puerto 9000.

Respuesta obtenida:

```
{"mensaje":"Hola, Equipo","desde":"dulcehogar:9000"}
```

Esta respuesta, obtenida al realizar la solicitud a través del dominio configurado en Apache, junto con la configuración del Virtual Host y las pruebas de conectividad, permite comprobar que ambos componentes se comunican correctamente. Apache recibe la petición y la reenvía a Express, que procesa la solicitud y devuelve la respuesta JSON al cliente.

Captura: fase3-curl-exitoso.png



FASE 4 - HTTPS CON MKCERT

COMANDOS EJECUTADOS (WSL2 / UBUNTU)

```
# 1. Instalar mkcert
sudo apt update
sudo apt install -y libnss3-tools wget
wget -O mkcert
https://github.com/FiloSottile/mkcert/releases/download/v1.4.4/mkcert-v1.4.4-
linux-amd64
chmod +x mkcert
sudo mv mkcert /usr/local/bin/mkcert
mkcert -install

# 2. Generar certificado
cd ~
mkcert dulcehogar.local
sudo mkdir -p /etc/apache2/ssl/dulcehogar.local
sudo mv dulcehogar.local.pem dulcehogar.local-key.pem
/etc/apache2/ssl/dulcehogar.local/
sudo chmod 600 /etc/apache2/ssl/dulcehogar.local/*.pem
ls -la /etc/apache2/ssl/dulcehogar.local/

# 3. Editar ports.conf
sudo nano /etc/apache2/ports.conf

# 4. Crear Virtual Host SSL
sudo nano /etc/apache2/sites-available/dulcehogar.local-ssl.conf
sudo a2enmod ssl
sudo a2ensite dulcehogar.local-ssl.conf
sudo apache2ctl configtest
sudo systemctl restart apache2
sudo ss -tlnp | grep apache2

# 5. Importar CA en Windows (copiar el certificado raíz al escritorio)
cp $(mkcert -CAROOT)/rootCA.pem /mnt/c/Users/isvij/Desktop/

# 6. Windows PowerShell (Administrador)
certutil -addstore -f "ROOT" C:\Users\isvij\Desktop\rootCA.pem

# 7. Probar HTTPS
curl -I https://dulcehogar.local:8900/
curl "https://dulcehogar.local:8900/api/saludo?nombre=Equipo"
curl
"https://dulcehogar.local:8900/api/cotizacion?nombre=Isvi&telefono=7777&producto=
pastel"
```

CONFIGURACIÓN: PORTS.CONF

```
Listen 80
Listen 8085
Listen 8090
Listen 8900

<IfModule ssl_module>
        Listen 443
        Listen 4443
</IfModule>

<IfModule mod_gnutls.c>
        Listen 443
</IfModule>
```

CONFIGURACIÓN: DULCEHOGAR.LOCAL-SSL.CONF

```
<IfModule mod_ssl.c>
<VirtualHost *:8900>
    ServerName dulcehogar.local
    DocumentRoot /var/www/dulcehogar.local

    SSLEngine on
    SSLCertificateFile /etc/apache2/ssl/dulcehogar.local/dulcehogar.local.pem
    SSLCertificateKeyFile /etc/apache2/ssl/dulcehogar.local/dulcehogar.local-
key.pem

    ProxyPreserveHost On
    ProxyPass /api http://localhost:9000/api
    ProxyPassReverse /api http://localhost:9000/api
</VirtualHost>
</IfModule>
```

RESULTADOS OBTENIDOS

- mkcert -install: CA local creada e instalada.
- Certificado generado: dulcehogar.local.pem + dulcehogar.local-key.pem.
- CA importada en Windows: CertUtil completado correctamente.
- Puerto 8900 escuchando.
- HTTPS: HTTP/1.1 200 OK.
- Navegador: candado verde sin advertencia.

1. ¿Por qué certbot no funciona para dulcehogar.local?

Let's Encrypt, mediante Certbot, no emite certificados públicamente confiables para nombres internos como dulcehogar.local, ya que estos nombres no cumplen los requisitos necesarios para obtener un certificado

público. Los dominios terminados en .local se utilizan habitualmente en redes y entornos de desarrollo local.

Para solucionar esta situación, se utiliza **mkcert**, una herramienta que crea una Autoridad Certificadora (CA) local y genera certificados firmados por ella. Estos certificados pueden ser reconocidos como confiables por los equipos donde se haya instalado dicha CA, lo que permite utilizar HTTPS en un entorno de desarrollo sin necesidad de publicar la aplicación en Internet.

2. ¿Qué diferencia hay entre un certificado autofirmado y uno firmado por mkcert?

Un certificado autofirmado está firmado por su propio emisor y normalmente genera advertencias en el navegador cuando este no confía en él. En cambio, mkcert crea una Autoridad Certificadora (CA) local y utiliza esa autoridad para firmar los certificados generados. Cuando la CA está instalada como confiable y el certificado corresponde al dominio solicitado, el navegador puede aceptarlo sin mostrar advertencias de seguridad.

Ambas opciones pueden utilizarse en entornos locales, pero mkcert facilita la gestión de certificados confiables para el desarrollo sin necesitar un dominio público ni un certificado emitido por una autoridad certificadora comercial.

3. Reto: si desplegaran en un dominio público real (dulcehogar.com.sv), ¿qué cambiarían?

Si se desplegara en un dominio público real (dulcehogar.com.sv), se cambiaría:

1. Certificado: usar certbot con Let's Encrypt para obtener un certificado públicamente confiable.
2. Puerto: configurar HTTPS en el puerto estándar 443.
3. DNS: apuntar el dominio real a la IP pública del servidor.
4. ServerName: cambiar dulcehogar.local por dulcehogar.com.sv.
5. Redirect: configurar redirección de HTTP (80) a HTTPS (443).
6. Renovación: configurar y comprobar el mecanismo de renovación automática del certificado, teniendo en cuenta su período de validez.

4. Capturen pantalla de https://dulcehogar.local:8900/api/saludo?nombre=Equipo sin advertencia. (Recuerden incluir el puerto :8900 en el navegador

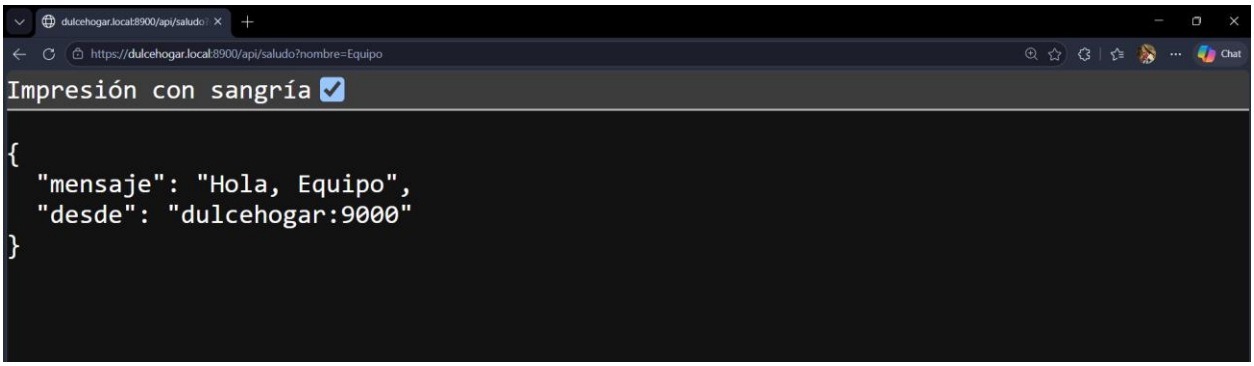
- URL: https://dulcehogar.local:8900/api/saludo?nombre=Equipo
- Candado verde (sin advertencia).
- Respuesta JSON: {"mensaje":"Hola, Equipo","desde":"dulcehogar:9000"}
- Visor de certificados con "mkcert development CA".

Esta evidencia permite comprobar que la conexión HTTPS funciona sin advertencias en el navegador y que la API responde a través del Virtual Host configurado en Apache.

HTTPS sin advertencia en el navegador.

Captura: fase4-https-sin-advertencia.png

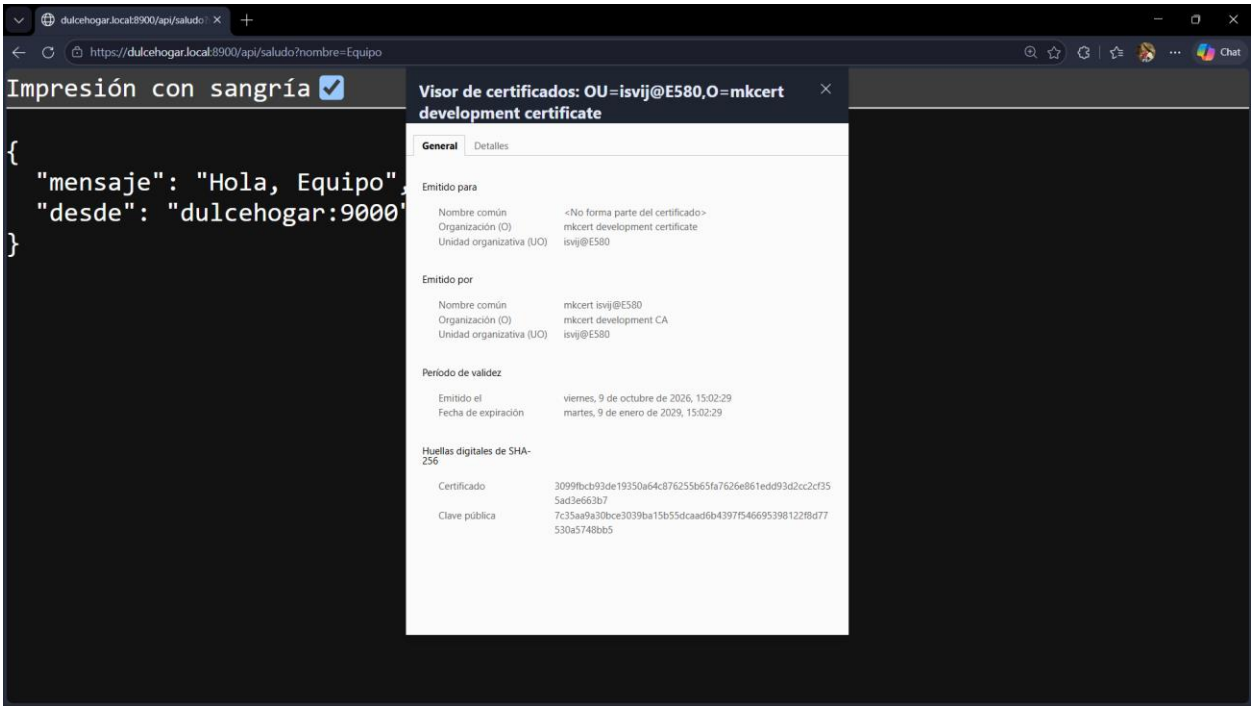
La captura muestra:



Detalle del certificado emitido por mkcert development CA.

Como evidencia adicional, se muestra el visor de certificados del navegador, donde se confirma que:

- El certificado fue emitido por "mkcert development CA".
- La organización es "mkcert development certificate".
- La unidad organizativa corresponde al equipo (isvij@E580).
- El período de validez es del 9 de octubre de 2026 al 9 de enero de 2029.
- Las huellas digitales SHA-256 del certificado y de la clave pública están registradas.



REPOSITORIO GITHUB

Enlace del repositorio

Oscxr29/gsw-practico-unidad2-equipo03

ESTRUCTURA PROPUESTA DEL REPOSITORIO

```
Laboratorio 2/
|-- README.md
|-- informe-final.pdf
|-- fase1-dimensionamiento.txt
|-- fase2-apache.txt
|-- fase3-virtualhost.txt
|-- fase4-https.txt
|-- config/
|          |-- dulcehogar.local.conf
|          |-- dulcehogar.local-ssl.conf
|          |-- ports.conf
|-- capturas/
|          |-- fase2-apache-activo.png
|          |-- fase3-curl-exitoso.png
|          |-- fase4-https-sin-advertencia.png
```

CONCLUSIONES

1. El dimensionamiento permitió concluir que 2 vCPU y 4 GB de RAM son suficientes para el tráfico normal de Dulce Hogar (unas 500 visitas diarias) y que el factor pico de 6 exige confirmar la capacidad con una prueba de carga antes de cada fecha festiva.
2. El escalado vertical temporal, combinado con caché de archivos estáticos, es la estrategia más adecuada para una PyME sin personal de TI, ya que evita el costo permanente de un balanceador y la complejidad de administrar varios servidores. La nube (VPS) resultó la opción preferible frente al servidor local.
3. Apache quedó instalado y operativo en WSL2, y la verificación con systemctl, curl y apache2ctl configtest confirmó el servicio activo, la página por defecto y la sintaxis de configuración correcta. configtest se estableció como paso obligatorio antes de cada recarga o reinicio.
4. El Virtual Host con Reverse Proxy permitió publicar el sitio estático y la API Express bajo un mismo dominio (dulcehogar.local), con el backend protegido detrás de Apache y gestionado como servicio systemd con reinicio automático ante fallos.

5. La respuesta JSON obtenida a través del dominio configurado comprobó la integración entre Apache (puerto 80) y Express (puerto 9000), validando que la cadena cliente, proxy y backend funciona de extremo a extremo.
6. HTTPS con mkcert permitió contar con un entorno local confiable: al instalar la CA local en Windows, el navegador mostró el candado sin advertencias en el puerto 8900. Para producción se requiere un dominio público, Let's Encrypt con Certbot, el puerto 443, redirección de HTTP a HTTPS y renovación automática.

ROL DEL DOCUMENTADOR / QA

Recolección de evidencias, la ejecución de pruebas de integración y la validación de HTTPS realizadas durante el laboratorio.

A. CAPTURAS DE EVIDENCIA

N.	Archivo	Fase	Qué demuestra
1	fase2-apache-activo.png	2	¡Apache activo y página por defecto “It works!" servida desde /var/www/html.
2	fase3-curl-exitoso.png	3	Respuesta JSON de la API Express obtenida a través del Virtual Host de Apache.
3	fase4-https-sin-advertencia.png	4	Conexión HTTPS en el puerto 8900 con candado verde, respuesta JSON y visor de certificado "mkcert development CA".

Nota: la Fase 1 es de análisis y no requiere captura.

B. PRUEBAS DE INTEGRACIÓN

ID	Prueba	Resultado esperado	Resultado obtenido
PI-01	Apache escucha en el puerto 80 (curl http://localhost).	Página por defecto de Apache.	OK: "Apache2 Ubuntu Default Page - It works!".
PI-02	Sintaxis de configuración (apache2ctl configtest).	Syntax OK.	OK: Syntax OK (Fases 2 y 3).
PI-03	Backend Express activo (systemctl status dulcehogar).	Servicio active en el puerto 9000.	OK: active.
PI-04	API vía proxy HTTP: /api/saludo?nombre=Equipo.	JSON con mensaje y origen dulcehogar:9000.	OK: {"mensaje":"Hola, Equipo","desde":"dulcehogar:9000"}.

ID	Prueba	Resultado esperado	Resultado obtenido
PI-05	API vía proxy HTTP: /api/cotizacion con nombre, telefono y producto.	JSON con cotizacionId, datos, fecha y estado pendiente (según index.js).	Ejecutada. Registrar aquí la salida obtenida: {"cotizacionId":1791672465603,"nombre":"Isvi","telefono":"6767","producto":"pastel","fecha":"2026-10-10","estado":"pendiente"}.
PI-06	Cabeceras HTTPS (curl -I https://dulcehogar.local:8900/).	HTTP/1.1 200 OK.	OK: HTTP/1.1 200 OK.
PI-07	API vía HTTPS: /api/saludo?nombre=Equipo en el puerto 8900.	Mismo JSON que en HTTP.	OK: respuesta JSON correcta.

C. VALIDACIONES DE HTTPS

Verificación	Criterio de aceptación	Estado
CA local instalada (mkcert -install).	CA local creada e instalada.	Cumple
Certificado emitido para dulcehogar.local.	Archivos .pem y -key.pem generados.	Cumple
Permisos de la clave privada.	chmod 600 sobre los archivos .pem.	Cumple
CA confiable en Windows.	CertUtil completado correctamente.	Cumple
Puerto 8900 en escucha.	ss -tlnp muestra apache2 en 8900.	Cumple
Respuesta HTTPS.	HTTP/1.1 200 OK.	Cumple
Navegador sin advertencia.	Candado verde y certificado "mkcert development CA".	Cumple